

29.11.2024г.

OOP 2

Лекция 7

self - така се кръщава първият аргумент на методите на клас

Когато методът е декориран със @staticmethod - не приемаме self като аргумент

→ все едно метода е извън класа ни

Когато методът е декориран с @classmethod - първият метод ще е самият клас - cls

class - запазена дума

```
class Spam:
```

```
    def method(self):    → public
        pass
```

```
    def _method(self): → protected
        pass
```

```
    def __method(self): → private (name mangling)
        pass
```

```
class Spam:
```

```
    def method(self):
        pass
```

```
spam = Spam()
```

```
object1 = Spam.method
```

→ Unbound method (отаква 1 аргумент)
извиква се върху самия клас

```
object2 = spam.method
```

→ Bound method (не отаква аргументи)
извиквам го върху
вече създаден обект от класа

--call-- (self) → Инстанциите могат да се
викат като функции

del → с ключовата дума del можем да изтриваме
атрибути на обект.

Ако искаме да достъпваме аргументи като ползваме индекси:

• Дефинираме --getitem-- → за да може класът ни да
↓
в имплементацията } поддържа достъп от вида
и на return object[index]

• Дефинираме --setitem-- → на съществуващ вече индекс
↓
нямаме return променяме стойността на обекта

--getattr-- → търсим го в случай, че няма дефиниран
атрибут с търсеното име

--setattr-- → извиква се винаги при присвояване на
нова стойност

АКО Е self.name = value → безкрайна рекурсия

затова:

object.--setattr--

--getattribute-- → извиква се при всяко търсене на
атрибут

object.--getattribute--

Всеки обект може да го разглеждаме като съвкупност от две неща:

- Речник, съдържащ атрибутите на обекта (`__dict__`)
- Връзка към класа на обекта (`__class__`)

Функциите и променливите, дефинирани в тялото на класа, са атрибути на класа.

Когато срещнем `some.thing` правим следните неща:

- 1) `some.__dict__['thing']` → връща стойност
- 2) `some.__class__.thing` → функция ✓ → bound method
→ функция X → връща директно
- 3) `some.__getattr__('thing')`
- 4) иначе грешка

Наследяване

Всички класове в Python наследяват обект

Един клас може да наследява няколко класа (тогава ще гледане от първия наследен към последния)

- Можем да използваме атрибути и методи на родителя
- Можем да "презаписваме" методи на родителя
- Детето има достъп и търси в атрибутите на родителя
- Ако детето няма конструктор, използва се родителския
- Ако детето има конструктор, използва се неговият
- Можем да използваме и родителския конструктор
`super().__init__(...)`
→ намира конструктора на родителя

Наследяване

Връзката между
класовете е IS
E

Композиция

Връзката между
класовете е HAS
ИМА

Mixins

↳ класове, които изпълняват функционалност